



AGENTS.md — Orvio Engineering Contract for AI Coding Agents

Version: 0.1

Last updated: 2026-09-28

Owner: Engineering (Code X)

Applies to: All AI coding agents (Codex, Claude Code, Cursor, Gemini, Kilo Code, etc.)

0. Purpose

This file is the **engineering constitution** for AI agents working in the Orvio repository. It tells you how to safely understand, modify, test, and extend this codebase without breaking architecture, security, or product behavior.

Chain of truth:

PRD → architecture.md → AGENTS.md → Implementation → Tests → Review

1. Read Before You Code

Rule: Never start coding immediately.

Before any change:

1. Inspect repository root.
2. Identify applications (apps/web, apps/api), packages, config, docs.
3. Locate the relevant module (e.g., inventory, pos, orders, payments, whatsapp).
4. Search for existing implementations (hooks, services, components, types).
5. Confirm where the change belongs (frontend feature, backend module, shared types).

If you cannot locate the relevant module, ask before proceeding.

2. Source of Truth Hierarchy

When requirements conflict, use this priority:

1. **Explicit user instruction**
2. **Current project architecture** (architecture.md)
3. **Current PRD / product requirements** (docs/PRD.md)

4. Existing business rules
5. Existing code behavior
6. AGENTS.md conventions
7. General engineering assumptions

Important: If existing code contradicts documented architecture, do not blindly overwrite. Investigate whether it is intentional, legacy, or incomplete. Make the smallest appropriate change.

3. Context Discovery

Minimum discovery steps:

```
Repository root
↓
Package configuration (package.json, pnpm-lock.yaml)
↓
Application structure (apps/, packages/)
↓
Relevant feature/module (e.g., apps/api/src/modules/inventory)
↓
Related services, types, API endpoints
↓
Related tests
↓
Configuration (tsconfig, vite, fastify, prisma)
```

Search before creating:

- Before creating `useAuth()`, search for existing auth hooks/services.
- Before creating `ApiClient`, search for existing API clients.
- Before creating `UserService`, search for existing user services.

Never create a duplicate abstraction without proving an existing one cannot be reused.

4. No Duplicate Implementations

Strict rule: Do not create duplicate:

- Components
- API clients
- Hooks
- Services
- Utilities
- Validation schemas
- Types

- Authentication/authorization logic
- State management
- Database access patterns
- Feature implementations

If similar functionality exists:

1. Find it.
2. Understand it.
3. Determine whether it should be reused.
4. Extend if appropriate.
5. Refactor only if necessary.

If two implementations exist, investigate which is canonical before creating a third.

5. Canonical Implementation Rule

Every important capability should have a clearly identifiable canonical implementation:

- Authentication
- Authorization
- API client
- Error handling
- Notifications
- Logging
- Validation
- Database access
- User state
- Permissions
- File uploads

Agents must prefer the canonical implementation. If multiple competing implementations exist, document the discovery and determine which one the architecture intends to use.

6. Change Scope

Keep changes proportional to the task.

If asked to fix "login redirect", do not rewrite "authentication architecture" unless the investigation proves the architecture itself is the cause.

Avoid:

- Unrelated refactors
- Formatting entire directories
- Renaming unrelated files
- Dependency upgrades unrelated to the task
- Architecture rewrites
- Large migrations without explicit approval

Prefer: Smallest correct change over largest possible improvement.

7. Before Implementation Checklist

Before writing code, determine:

- What is being changed?
- Why is it being changed?
- Where does the change belong?
- What existing code is affected?
- What dependencies exist?
- What could break?
- What tests cover this behavior?
- What new tests are required?
- Are there security implications?
- Are there authorization implications?
- Are there data migration implications?
- Does this change affect API contracts?
- Does this change affect frontend/backend contracts?

8. Plan Before Complex Changes

For non-trivial tasks, create a short implementation plan:

1. Files/modules to inspect
2. Root cause or requirement
3. Files likely to change
4. Implementation approach
5. Tests required
6. Potential risks

Do not create a massive plan for trivial changes.

9. Requirement Interpretation

Do not invent requirements.

If a requirement says "Users can manage their team," do not automatically assume delete, change roles, transfer ownership, bulk import, etc., unless documented or logically required.

If ambiguity materially affects implementation: **stop and ask for clarification.**

If ambiguity has a safe, conventional interpretation that does not affect architecture or business behavior: proceed and document the assumption.

10. Frontend Rules

Use React according to the project's established architecture.

- **Components:** Primarily presentation and UI interaction.
- **Hooks:** Reusable UI/application behavior.
- **Services/API clients:** Communication with backend.
- **State stores:** Only state that genuinely needs shared/persistent client state.
- **Server-state libraries:** Server/API state (TanStack Query).
- **Utilities:** Reusable pure functionality.

Do not put large business workflows directly inside React components.

11. React Component Rules

Avoid giant components. If a component is responsible for data fetching, business logic, permission evaluation, form validation, state management, API mutation, and complex rendering all at once, investigate whether responsibilities should be separated.

Do not split components merely to satisfy an arbitrary line count. Optimize for meaningful responsibility boundaries.

12. State Management Rules

Do not put all application state into a global store. Use the smallest appropriate state scope.

Prefer:

```
Component state
↓
Feature state
↓
URL state
↓
Server/query state
```

↓
Global state

Only use global state when multiple unrelated parts of the application genuinely need it. Avoid storing duplicate copies of server data.

13. API Rules

Frontend code must communicate with backend through the project's established API layer.

Do not:

- Directly access the database from the frontend
- Duplicate API clients
- Hardcode production URLs
- Bypass authentication mechanisms
- Bypass authorization
- Create undocumented endpoints
- Silently change API contracts

If an API contract must change:

1. Identify all consumers.
2. Update backend.
3. Update frontend.
4. Update types/contracts.
5. Update tests.
6. Verify compatibility.

14. Backend Rules

Use the established Node.js + Fastify architecture.

Keep responsibilities separated:

```
HTTP Route
↓
Request Validation
↓
Authentication
↓
Authorization
↓
Application/Service Layer
↓
Domain Logic
```

↓
Repository/Data Access
↓
Database

Avoid putting business logic directly inside route handlers. Avoid database queries scattered throughout unrelated modules.

15. Database Rules

Agents must understand the existing database architecture before changing it.

Before changing the database:

- Inspect schema
- Inspect relationships
- Inspect indexes
- Inspect migrations
- Inspect constraints
- Inspect existing queries
- Inspect affected API behavior

Do not:

- Delete data casually
- Rename columns without checking consumers
- Remove fields without migration analysis
- Introduce duplicate entities
- Bypass constraints
- Make destructive schema changes without explicit authorization

Any destructive migration must be treated as high risk.

16. Authentication Rules

Authentication is a security boundary.

Agents must not casually modify:

- Session behavior
- Token generation
- Token validation
- Password handling
- Cookie settings

- Refresh token behavior
- OAuth callbacks
- Logout behavior

Before changing authentication, inspect the entire flow:

```
Login
↓
Session creation
↓
Token issuance
↓
Client state
↓
Authenticated request
↓
Token/session validation
↓
Refresh
↓
Logout/revocation
```

A change that appears local may affect the entire authentication lifecycle.

17. Authorization Rules

Authorization must be enforced server-side. Frontend permission checks are for UX only.

Never assume: `if (!canEdit) hideButton()` is sufficient security. The backend must independently verify authorization.

Agents must consider:

- User role
- Resource ownership
- Organization/tenant
- Branch/location
- Permission
- Resource state

before allowing sensitive operations.

18. Multi-Tenant Security

If the system is multi-tenant:

Every tenant-scoped request must establish and validate tenant context.

Agents must actively consider: Can User A access Tenant B's data? for every data access path.

Never trust:

- Client-provided tenant IDs
- Client-provided organization IDs
- Hidden form fields
- URL parameters alone

without server-side authorization.

Tenant isolation is a security requirement.

19. Loading State Rules

Use loading states intentionally.

Prefer:

- **Skeleton loading:** Pages, tables, cards, dashboards, lists, content-heavy components.
- **Spinner/loading indicator:** Small localized actions, buttons, inline operations, short mutations.
- **Full-page loading:** Only when the application cannot meaningfully render anything before required initialization completes.

Avoid: Blank screens, multiple competing spinners, layout jumps, loading indicators that appear for trivial operations.

Preserve existing loading conventions where they exist.

20. Error Handling

Errors must be handled intentionally.

Every significant operation should consider:

- Success
- Validation failure
- Authentication failure
- Authorization failure
- Not found

- Conflict
- Network failure
- Server failure
- Timeout
- Unexpected error

Do not:

- Silently swallow errors
- Expose internal implementation details to users
- Display raw backend stack traces

Use the project's centralized error handling system.

21. Empty States

Do not treat an empty dataset as an error. Design meaningful empty states.

Example:



No team members yet.
Invite your first team member to get started.
[Invite Member]

Empty states should explain:

- What is empty
- Why it matters
- What the user can do next

22. Validation

Validation should occur at the appropriate boundaries.

- **Frontend validation:** User experience, immediate feedback.
- **Backend validation:** Security, data integrity, business rules.

Never rely exclusively on frontend validation.

23. Security

Before completing a task, consider:

- Authentication
- Authorization
- Input validation

- Injection
- XSS
- CSRF
- SSRF
- Sensitive data exposure
- Session security
- Rate limiting
- File upload security
- Tenant isolation
- Secrets

Do not weaken security to make implementation easier.

Never hardcode:

- Passwords
- API keys
- Tokens
- Private credentials
- Production secrets

24. Environment Variables

Never hardcode environment-specific configuration. Use the project's established configuration system.

Separate:

- Development
- Test
- Staging
- Production

Never expose server-only secrets to frontend code.

25. Dependency Rules

Before installing a package:

1. Search the repository for an existing solution.
2. Check whether native functionality is sufficient.
3. Check whether an existing dependency already provides the capability.

4. Determine maintenance/security implications.
5. Determine whether the dependency is necessary.

Do not install packages simply because they make one task slightly easier.

When adding a dependency, document why it exists.

26. Type Safety

Prefer TypeScript where the project uses TypeScript.

Avoid: any unless there is a documented reason.

Prefer:

- Explicit types
- Inferred types where clear
- Shared contracts
- Runtime validation at external boundaries

Do not create excessive types that merely wrap obvious primitives.

27. API Contract Safety

Treat API contracts as shared interfaces between systems.

Before modifying an API:

- Search frontend consumers
- Search backend consumers
- Search tests
- Search documentation
- Search generated types

Consider backward compatibility.

Never silently rename:

- fields
- endpoints
- parameters
- status codes

without checking consumers.

28. Testing Requirements

Every implementation should be validated appropriately.

At minimum determine whether the change requires:

- Unit tests
- Component tests
- Integration tests
- API tests
- End-to-end tests
- Security tests

Tests should focus on behavior rather than implementation details.

For bug fixes: Add a regression test whenever practical.

29. Test Before Claiming Completion

Never report "Done" based solely on code changes.

Run the appropriate verification. At minimum where applicable:

- Type check
- Lint
- Unit tests
- Integration tests
- Build
- Relevant E2E tests

Do not claim a test passed if it was not actually executed. If a check cannot be run, state exactly why.

30. Do Not Mask Failures

Never:

- Disable a test to make CI pass
- Suppress an error without understanding it
- Add arbitrary retries to hide failures
- Increase timeouts without investigation
- Ignore TypeScript errors
- Disable lint rules globally
- Remove failing assertions

- Delete tests because implementation is difficult

Fix the underlying issue or clearly report the limitation.

31. Debugging Process

When something fails:

1. Reproduce the failure.
2. Collect the actual error.
3. Identify the failing layer.
4. Trace the data/control flow.
5. Find the root cause.
6. Implement the smallest correct fix.
7. Add or update regression tests.
8. Run the relevant validation.

Do not patch symptoms repeatedly without understanding the root cause.

32. Logging and Debugging

Use the existing logging system.

Do not add random `console.log()` throughout production code unless the project convention allows it.

Logs should provide useful diagnostic context without exposing:

- Passwords
- Tokens
- Secrets
- Sensitive personal data

33. Performance

Do not optimize blindly. First identify the bottleneck.

Consider:

- Network requests
- Database queries
- Rendering
- Bundle size
- Memory

- Large lists
- Repeated computation
- API latency

Avoid premature optimization that makes code harder to understand.

34. Refactoring Rules

Refactoring is allowed when it materially improves the requested change.

However: Do not combine feature implementation + unrelated architecture rewrite + massive formatting change + dependency upgrades into one task.

Keep changes focused.

35. File and Naming Conventions

Follow existing project conventions.

Before creating a new file, inspect neighboring files. Match:

- Naming
- Directory structure
- Export style
- Component patterns
- Service patterns
- Test naming
- Type naming

Consistency with the repository is more important than personal preference.

36. Documentation Rules

When behavior changes significantly, update relevant documentation.

Documentation should explain:

- Why something exists
- Important constraints
- Non-obvious behavior
- Architectural decisions
- Integration requirements

Do not document obvious code line-by-line.

37. Git / Change Hygiene

Keep changes reviewable.

Avoid:

- Generated files unless required
- Temporary files
- Debug artifacts
- Unrelated formatting
- Build output
- Secrets
- Local configuration

Do not modify files unrelated to the task without justification.

If Git is available, inspect the diff before completion.

38. Migration Safety

For changes involving data/schema:

Before migration:

- Understand existing data
- Understand consumers
- Determine backward compatibility
- Determine rollback strategy

Prefer additive migrations where possible:

```
Add new column
↓
Deploy compatible code
↓
Backfill
↓
Switch reads/writes
↓
Remove old field later
```

Do not perform destructive migrations casually.

39. Backward Compatibility

Before changing existing behavior, identify:

- Existing users
- Existing data
- Existing API consumers
- Existing URLs
- Existing integrations
- Existing permissions
- Existing workflows

Do not assume the current implementation has no consumers simply because they are not visible in the current feature.

40. Agent Communication

When starting a complex task, provide a concise summary of:

- What you discovered
- What you believe the problem is
- What you plan to change

When finishing, report:

- **Changed:** Files/modules changed and why.
- **Verified:** Tests/checks actually executed.
- **Not verified:** Anything that could not be tested.
- **Risks:** Any remaining concern.
- **Follow-up:** Only genuinely necessary follow-up work.

Do not produce long irrelevant summaries.

41. When to Ask for Clarification

Ask the developer before proceeding when:

- Two requirements conflict
- A destructive change is required
- A business rule is ambiguous
- A security boundary is unclear
- A database migration could cause data loss
- A major architectural decision is required

- Multiple valid interpretations produce materially different behavior
- The requested behavior conflicts with the documented architecture

Do not ask unnecessary questions when the repository already provides the answer.

42. When Not to Ask

Do not ask for clarification about things that can be determined from:

- Existing code
- Existing architecture
- Existing patterns
- Existing tests
- PRD
- Configuration
- Established conventions

Investigate first.

43. Existing Code Is Evidence

Treat the existing repository as evidence.

Before creating a new abstraction, search for:

- Existing implementation
- Existing helper
- Existing service
- Existing component
- Existing hook
- Existing schema
- Existing endpoint
- Existing test

The agent should be able to answer: "Why does this new code need to exist?" before creating it.

44. Do Not Trust Names Alone

A file named `LegacyUserService` may still be actively used. A file named `NewAuthService` may not actually be canonical.

Determine usage through:

- Imports

- References
- Routes
- Tests
- Runtime flow
- Configuration

Never delete or replace code based solely on its filename.

45. Feature Ownership

Every feature should have a clear ownership boundary.

Before modifying a feature, determine:

- Which module owns it
- Which service owns its business logic
- Which API owns its data
- Which UI owns its presentation
- Which permissions control it

Avoid cross-module leakage.

46. Cross-Module Changes

If a change affects multiple modules:

- Document the dependency.

Example:

```
Authentication
  ↓
User
  ↓
Organization
  ↓
Membership
  ↓
Permissions
  ↓
Feature
```

Do not modify downstream behavior without understanding upstream dependencies.

47. Feature Flags

If the project uses feature flags:

- Use the existing feature-flag mechanism.
- Do not invent a second feature-flag system.

Feature flags should have:

- Clear purpose
- Owner
- Default behavior
- Rollout behavior
- Removal plan

48. Deployment Safety

Before changing production-sensitive functionality, consider:

- Environment variables
- Database migrations
- Existing sessions
- Existing users
- API compatibility
- Rollback
- Monitoring
- Error rates

Production changes must be reversible where reasonably possible.

49. Performance Regression Check

For changes involving:

- Database
- API
- Large lists
- Authentication
- Search
- Reports
- Dashboards

Consider whether the change could introduce:

- N+1 queries
- Excessive requests
- Large payloads
- Unnecessary rerenders
- Memory leaks
- Slow queries

50. Final Pre-Completion Checklist

Before declaring a task complete:

Understanding

- ☐ I understood the existing implementation.
- ☐ I searched for existing functionality.
- ☐ I identified the correct module.

Implementation

- ☐ I followed the architecture.
- ☐ I reused existing abstractions where appropriate.
- ☐ I avoided duplicate functionality.
- ☐ I avoided unnecessary dependencies.
- ☐ I kept the scope focused.

Security

- ☐ Authentication remains correct.
- ☐ Authorization is enforced server-side.
- ☐ Tenant isolation is preserved where applicable.
- ☐ No secrets were introduced.
- ☐ Input is validated.

UX

- ☐ Loading states are handled.
- ☐ Empty states are handled.
- ☐ Error states are handled.
- ☐ Success states are handled.
- ☐ Responsive behavior is preserved.

Testing

- ☐ Relevant tests were run.

- [] Regression tests were added where appropriate.
- [] Type checking passed.
- [] Lint passed where applicable.
- [] Build passed where applicable.

Quality

- [] No unrelated files were changed.
- [] No debug code remains.
- [] No temporary files remain.
- [] API contracts remain consistent.
- [] Documentation was updated where necessary.

51. Special Rule for AI Agents

The agent must never assume that generating more code means completing the task better.

Prefer: Correct + minimal + maintainable over Large + complicated + impressive.

The agent's responsibility is not merely to write code. Its responsibility is to preserve and improve the integrity of the entire system.

52. Project-Specific Information

Repository structure (as of 2026-09-28):

```

orvio/
├── apps/
│   ├── web/           # React + Vite frontend
│   └── api/           # Node.js + Fastify backend
├── packages/
│   ├── shared/        # Shared types, schemas
│   └── config/        # ESLint, TS configs
├── docs/
│   ├── PRD.md
│   └── architecture.md
├── infra/              # Deployment configs (future)
└── package.json

```

Package manager: pnpm (assumed; confirm from package.json)

Important commands (to be confirmed from repo):

```

# Install
pnpm install

# Development

```

```
pnpm dev          # May run both web and api concurrently

# Type check
pnpm typecheck

# Lint
pnpm lint

# Test
pnpm test

# Build
pnpm build
```

(Agents must verify these commands from actual `package.json` before using.)

53. Important Project Files

Agents should inspect these files before making major changes:

- **Architecture:** `docs/architecture.md`
- **Product requirements:** `docs/PRD.md`
- **Frontend:** `apps/web/`
- **Backend:** `apps/api/`
- **Database schema:** `apps/api/prisma/schema.prisma`
- **Authentication:** `apps/api/src/modules/auth/`
- **Authorization:** `apps/api/src/middleware/auth.ts` (or equivalent)
- **API layer:** `apps/api/src/modules/*/routes.ts`
- **Frontend services:** `apps/web/src/services/`
- **Frontend features:** `apps/web/src/features/`
- **Testing:** `apps/*/tests/` or `apps/*/__tests__/`

54. Final Requirement

This file is binding for all AI coding agents working in this repository.

If an agent cannot satisfy these requirements, it must:

1. Stop.
2. Report what it cannot satisfy and why.
3. Ask for clarification.

The goal is not to complete tasks quickly. The goal is to complete tasks correctly, safely, and maintainably while preserving the integrity of the Orvio codebase.

End of AGENTS.md